Montgomery Modular exponentiation on FPGA

Anane Nadjia[(1)], Anane Mohamed [(2)] and Issad Mohamed[(1)]
[(1)] CDTA (Centre de Développement des Technologies Avancées), BP 17, Baba Hassen)
[(2)] ESI (Ecole nationale Supérieure d'Informatique), BP 68M Oued Smar.
Alger, Algérie
e-mail: anane@cdta.dz

***Abstract*—Modular exponentiation is the main operation of RSA-based public-key cryptosystems. It is implemented by repeated modular multiplications which are time consuming for large operands. Accelerating the RSA requires reducing the number of modular multiplications, thus reducing the time to perform one modular multiplication.**

In this paper, we present an architecture designed to implement a fast modular exponentiation using the right to left binary method (R-L), which allows the parallel execution of modular operations "squares and multiplications". The fast modular multiplication used is based on Montgomery algorithm.

This architecture has been implemented on an FPGA circuit of Xilinx, the XC4VLX25-12ff668 of Virtex-4. The implementation results showed that the architecture computing 1024 bits modular exponentiation presents good performance in terms of speed and occupied area with the possibility of changing the key size by reprogramming the FPGA according to the security level and the performance to attain.

Keywords-component: *cryptosystem, modular exponentiation, Montgomery modular multiplication, RSA, FPGA.*

I. INTRODUCTION

The rapid and continuous development in communications through open networks such as Internet has created a growing need to encrypt sensitive or confidential data before their transfer.

The RSA (Rivest, Shamir, Adleman) is a secure and efficient public-key cryptosystem [1]. However, it is very intensive computing since it operates on large operands: 1024 bits or greater to provide sufficient security.

Software implementation of RSA is too slow for real-time applications [2]. However hardware implementation is required to achieve performances of such applications [3].

The key operation of RSA is the modular exponentiation which is used to encrypt a message M by using a public key e or recovers the original message from the encrypted message by using the private key d such as:

$$C = (M^e \bmod N), \quad M = (C^d \bmod N),$$

where N is the modulus of n bits.

Accelerating the RSA cryptosystem requires reducing the modular multiplications number hence the execution time of the modular multiplication.

The goal of this paper is to present an FPGA implementation of a fast modular exponentiation, by exploiting the parallelism offered by the R-L binary modular exponentiation, the high speed of Montgomery modular multiplication and the FPGA performances to provide more security with the possibility of changing the key size by reconfiguration of the FPGA depending on the desired security and performance.

The remainder of this paper is organized as follows. Section 2 presents the binary modular exponentiation. In section 3, the Montgomery modular multiplication is presented. Section 4 describes the architecture developed for computing the Montgomery modular exponentiation. Section 5 summarizes the implementation results of the architecture on an FPGA of Virtex-4. Finally, a conclusion and some perspectives are given.

II. BINARY MODULAR EXPONENTIATION

Modular exponentiation, realized by a series of modular multiplications, is very costly in computation time for large operands.

The simplest and easy method to compute 1024 bits modular exponentiation is the binary method [4], known as the "Square and multiply". It is based on scanning the bits of the binary exponent, then a squaring is performed at each step and depending on the scanned bit value, a subsequent multiplication is performed.

To increase the computation performances of this operation:

1. The R-L binary exponentiation is used, which scans the exponent bits from the least significant (LSB) and allows the squaring and multiplication to run in parallel [5].
2. The Montgomery algorithm is used, which replaces the division by the modulus by a series of addition and shift operations in order to reduce the execution time of the modular multiplication [6].

III. MONTGOMERY MODULAR MULTIPLICATION

Montgomery modular multiplication used to calculate the R-L binary exponentiation is fast and suitable for hardware implementation. It replaces the division by the modulus with simple right shifts, easy to implement but requiring some pre and post calculations.

978-1-4673-5292-5/12/$31.00 ©2012 IEEE

The application of Montgomery algorithm on two numbers A and B of n bits gives the following result:

$$C = MM(A, B) = A \times B \times r^{-1} \bmod N.$$

Where $r = 2^n$, and n is an integer with $2^{n-1} < N < 2^n$, such as gcd (r, N) = 1.

Since $r = 2^n$, it suffices to the modulus N to be an odd integer, where this condition is satisfied for the RSA.

The Montgomery algorithm used is without final subtraction represented by algorithm 1, which allows reduction of the execution time in plus of the occupied area.

Algorithm 1: Montgomery without final subtraction.

Inputs: $A = a_{n+1}a_n \ldots a_0$, $B = b_{n+1}b_n \ldots b_0$, $N = n_{n-1}n_{n-2} \ldots n_0$,
With A, B<2N, $a_{n+1} = b_{n+1} = 0$.
Output: $S_{n+2} = A \times B \times 2^{-n-2} \bmod N$

```
Begin
    S0=0
    For i from 0 to n+1 do
        1. qi = (Si+ai×B) mod 2
        2. Si+1=(Si+ ai×B+ qi ×N)/2
    End For.
End.
```

The subtraction has been omitted and replaced by two iterations which let the operand sizes equal to (n+1) instead of (n-1) [7].

The architecture of the Montgomery modular multiplication without final subtraction is represented on figure 1.

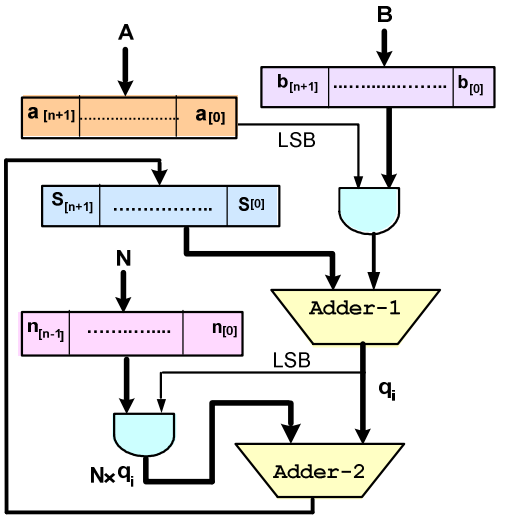


Figure 1. Montgomery modular multiplication architecture

IV. MONTGOMERY MODULAR EXPONENTIATION ARCHITECTURE

To adapt the Montgomery multiplication algorithm to the R-L binary modular exponentiation, we must make a mapping in order to enter in the Montgomery domain, where a series of modular multiplications are performed, then a re-mapping is required to recover the true result in the last iteration of the modular exponentiation out of the Montgomery domain.

Algorithm 2 shows the R-L binary modular exponentiation based on Montgomery modular multiplication.

Algorithm 2 : Montgomery based R-L binary exponentiation

Inputs: e, M, N, r^2; $e = (e_{n-1}\, e_{n-2} \ldots e_3\, e_2\, e_1\, e_0)_2$
Output: $C = M^e \bmod N$;

```
C = Montg (1, r 2, N), S = Montg (M, r 2, N);
Begin
  For i = 0 to n - 1 do
      Begin
        a. If (ei = 1) then C = Montg (C, S, N); [multiply]
        b. S = Montg (S, S, N); [square]
      End;
  C = Montg (C, 1, N).
End.
```

The proposed architecture for fast computing the binary modular exponentiation based on Montgomery multiplication is presented on figure 2.

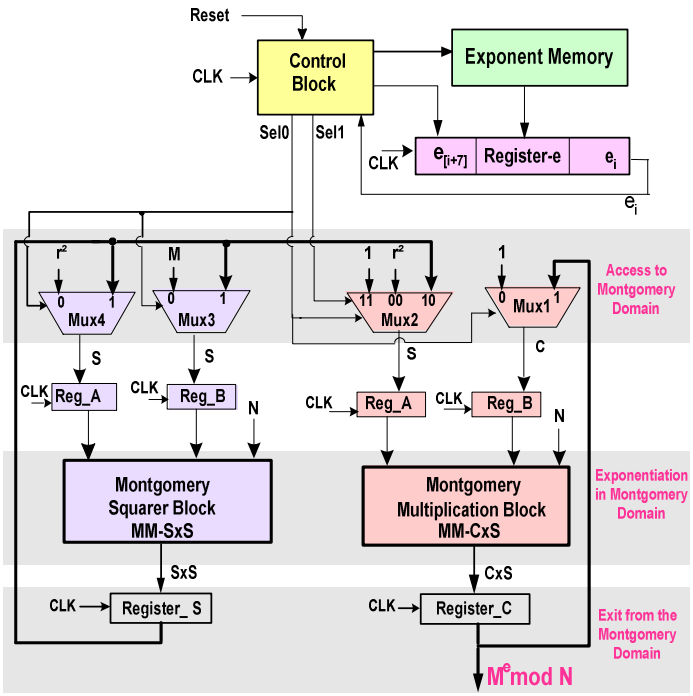


Figure 2. Architecture of the Montgomery R-L binary exponentiation

It consists of :

1. A control block to manage squaring and multiplication operations. It has as inputs the Reset and the clock CLK.
2. A memory to store the exponent e connected to a shift register for shifting the exponent bits.
3. Two Montgomery modular multiplication blocks MM-SS for squares and MM-CS for multiplications with two registers S and C at their outputs to store respectively the intermediate results of squares and multiplications.

4. Two registers A and B at the input of each multiplication block to store the inputs data (M, r^2) or the intermediate results.

5. Two multiplexers at the input of each multiplication block to select the inputs data (M, r^2,1) or the intermediate results.

Entering in the Montgomery domain needs to perform two Montgomery modular multiplications in parallel:

MM (M,r^2 (mod N)) and MM (1, r^2(mod N)),

which gives as outputs : M× r (mod N) and 1 × r (mod N).

In the Montgomery domain, multiplications and squares are performed in parallel by two Montgomery multipliers and successively until the exhaustion of all the exponent bits.

The multiplexers select the inputs (M×r mod N) and (1×r mod N) or the outputs of the multipliers according to the logic control to give the result (M^e × r mod N).

Exiting from the Montgomery domain means performing a last Montgomery multiplication in order to eliminate the r factor from the result to finally obtain (M^e mod N).

The two Montgomery multiplication blocks used are optimized in terms of execution time and area, where the blocks Select Ram (SRAM) of 18 Kbits available in the FPGA were used to store the operands.

Operands are introduced word per word which has an advantage to reduce the size of the adders and internal registers (1024 bits) to the size of the selected word. That has reduced the hardware and routing resources required to implement these registers.

The first eight bits of the exponent e are introduced into an eight bits serial shift register. Once the calculus is completed, eight successive bits are sent from the memory to take place in the shift register to continue the computing until the last bit of the exponent e.

Our architecture performs Montgomery modular exponentiation based on three phases:

- ***Phase 1:*** Entry in the Montgomery domain.
- ***Phase 2*****:** Computation of Montgomery modular exponentiation by modular squares and multiplications.
- ***Phase 3 :*** Exit from the Montgomery domain.

At each multiplication block, a pair of multiplexers is used to select the operands in the inputs of the two multiplication blocks according to the running phase of the exponentiation algorithm.

These multiplexers are controlled by the selection signals Sel0 and Sel1 as shown in Table 1.

TABLE I. SELECTION OF THE THREE EXPONENTIATION PHASES

Sel1	Sel0	Phase	Block MM-CS	Block MM-SS
0	0	1	Montg (1,r^2,N)	Montg (M,r^2,N)
0	1	2	Montg (C,S,N)	Montg (S,S,N)
1	1	3	Montg (C,1,N)	

V. IMPLEMENTATION RESULTS

The hardware architecture to perform the binary modular exponentiation in parallel was designed by using Xilinx ISE 9.2i and implemented on a Virtex-4 FPGA circuit of Xilinx, the XC4VLX25 12ff668 [8].

For best use of the resources offered by FPGAs, the exponent e of 1024 bits has been stored in Select RAM blocks of 18 K-bits, available on the FPGA to minimize the logic and routing resources necessary for the implementation of our architecture. This last was simulated using the ModelSim simulator PE 6.0 and the results were compared to those provided by Maple 12 tool [9], where two procedures were developed: one for key generation and encrypting the message and the other was reserved for the decomposition of the exponent e of 1024 bits in 8 bit words for storing them in the memory in order to reduce the size of the shift register.

We encrypt a message M represented by "mess" in figure 3 which is equal to:

mess:=10757843813889360743590595463473044150774486
22514571005685651967999358456333915068546583350371
52699326459104449152613250430281599897209937225878
62513872407589936594106183141435460637461318320428
54039261569700178938371760904881035693538331926792
17656677205119436507473692650971 8

The public key or the exponent e, the modulus N, the private key d and the constant r^2:=2**(2*1026) mod N were calculated by Maple 12 and are equals to:

e =11239,

N=82206061353103504512232870195738121018797639464
76971432239744157127909208618752807341343518216491
64870671593767502260078721884852495768439376898703
91450921580177437207188993844911556404643407098465
35134746178258983120388006834814186507938414323769
80519069092393832647740703266460859640751623240334
22113014621.

d=56678954482178045775006006864202749334875247638 8
02430490635979601018034040027329392373050024610369
05937213435451886300694026057231061223984991180760
41776859132542903127137598622589977233342978784811
04918467499480456837973144471337467918896301987183
09028154846263770002118974504022461436907938033122
6059975211.

r^2=74631413297914341936125959271525042613966027591
62738066614281346386171114676070332409719863370571
80743560150320716388568672873969588443173328297742
15338073167965039164622488648217077569665628757533
94010252124353551173106745204534528222961275025894
74409070843229521323332097266387442201686396581071
74997596445

These values have been loaded into the designed architecture which performs the modular exponentiation provided by our hardware to obtain the encrypted message C by: C= $(mess)^e$ mod (N).

At the same time this message was encrypted by Maple with computing the modular exponentiation by the following function:

C = (mess & ^ e) mod N,

which gives the following value for the encrypted message :

C-Maple :=3575596246652071170118289885250966388369145220924402971549147999524402195534176512791451192902409819125313768725620185970152570363454627674941507729156241021964801987223997045449502569281654961936154923397845084483423324618187445485127276676011839583337580177769165040119878715891274799142692038917469484859

We can find this value in the fifth line and the original message in the third line of the timing diagram of figure 3.

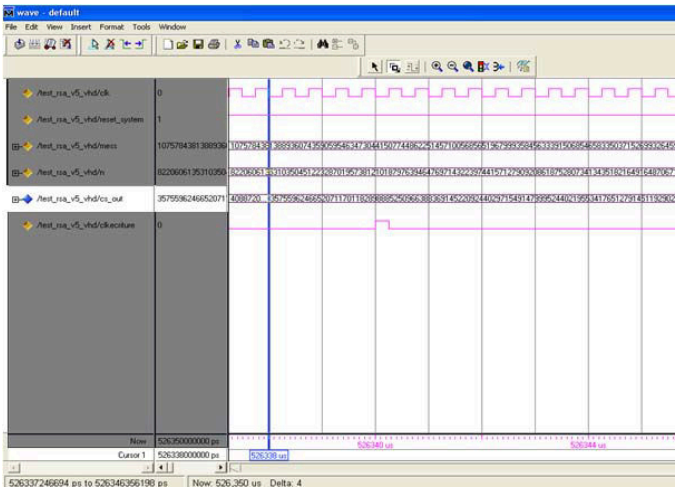


Figure 3. Results of encrypting the message mess

The computation results of the modular exponentiation provided by MAPLE for encrypting the message "mess" were equivalent to those provided by our architecture for computing the R-L binary modular exponentiation of the same message. That justifies the good working of our architecture.

The proposed architecture allows the computation of modular exponentiation for keys up to 1024 bits. The computation time for 1024-bits exponentiation is 85.63 ms.

$T_{total} = T \times 1026 \times 1026 = 85.63$ ms, where the first 1026 represents the number of Montgomery multiplications and the second 1026 represents the iterations number of the Montgomery multiplications.

The period $T = 2\times Tc = 81.35$ ns, where Tc is the time of critical path, Tc = 40, 675 ns.

The implementation results are summarized in table 2, where the numbers of slices, memory blocks and I/O blocks are given with the area occupancy percentage of our architecture on the FPGA.

TABLE II. OCCUPANCY RATE ON THE FPGA

Resources	Occupancy rate	%
Slices	10170 of 10752	94%
IOBS	3076 to 4486	68%
18Kbits BRAMS	2 of 72	3%

VI. CONCLUSION

This paper has described the architecture for computing the R-L binary modular exponentiation and its implementation on FPGA.

The binary modular exponentiation is computed by a series of squaring and multiplications.

Montgomery modular multiplication has been chosen to realize the two operations, which has greatly reduced the computation time of the modular exponentiation.

The architecture allows the parallel execution of squaring and multiplications.

The proposed architecture can support key sizes up to 1024 bits using the same circuits and making some changes in the control block with increasing the memory which is still available on the FPGA as only 3% were used.

The resources offered by FPGAs such as memory blocks and Carry Save Adders (CSA) have been used advantageously to speed-up the execution time of the modular exponentiation.

However this time can further be reduced by using the m-ary method which is a generalization of the binary method but instead of breaking the exponent into single bits, it breaks it in m-bits windows and then performs as many multiplications as there are non-zero windows that reduce the total number of operations needed by the modular exponentiation [10]. This method requires a certain amount of pre-computations ((m-1) powers of the message) which can be stored on the blocks Select Ram available on FPGAs.

REFERENCES

[1] Muhammad I. Ibrahimy, Mamun B.I. Reaz, Khandaker Asaduzzaman and Sazzad Hussain, "Hardware Prototyping of an Efficient Encryption Engine", International Journal of Electrical, Computer and Systems Engineering 4:4 2010.

[2] N. Nedjah L.M Mourelle, "Parallel computation of modular exponentiation for fast cryptography", Int. Journal of. High Performance Systems Architecture, Vol. 1, No. 1, 2007, Copyright © 2007 Inderscience Enterprises Ltd .

[3] Ersin Oksuzoglu and Erkay Savas. "Parametric, Secure and Compact Implementation of RSA on FPGA", International Conference on Recofigurable Computing and FPGAs, Dec.2008, pp.391-396.

[4] Ming-Der Shieh, Jun-Hong Chen, Hao-Hsuan Wu, and Wen-Ching Lin, "A New Modular Exponentiation Architecture for Efficient Design of RSA Cryptosystem", IEEE Trans. On VLS I Systems, Vol 16 N° 9 Sept. 2008.

[5] Colin D. Walter, "Right-to-Left or Left-to-Right Exponentiation?"□First International Workshop on Constructive Side-Channel Analysis and Secure Design COSADE 2010, pp. 40-46.

[6] Nathaniel Pinckney and David Money Harris, "Parallelized Radix-4 Scalable Montgomery Multipliers", Journal Integrated Circuits and Systems 2008; v.3 / n.1:39-45.

[7] Xia Hong, Hu Wenhao, Yan Jiangyu, "Design and Implementation of High-Performance Modular Exponentiation Arithmetic Unit", First International Conference on Information Science and Engineering (ICISE), 2009.

[8] "Virtex-4 FPGA User Guide", Xilinx .Dec. 2008.

[9] M.B.Monagan, K.O.Geddes, K.M. Heal, G. Labahn, S. M. Vorkoetter, J.McCarron, P.DeMarco, "Maple Introductory Programming Guide", Maplesoft, a division of Waterloo Maple Inc. 1996-2008.

[10] N. Nedjah L.M Mourelle, "High-performance SoC-based implementation of modular exponentiation using evolutionary addition chains for efficient cryptography", Applied soft computing , Elsevier journal, 11 (2011) 4302–4311.